

07 - Generics & Unit Of work

📅 09-09-2026 ⌚ Ongeveer 21 minuten 🗨️ Ongeveer 3172 woorden

In de voorgaande hoofdstukken hebben we onze architectuur opgezet met repositories. We hebben echter gezien dat naarmate we meer modellen toevoegen (zoals Product, Klant, Bestelling), onze repositories veel **gelijkaardige / duplicate code** gaan bevatten. Elke specifieke repository implementeert immers opnieuw de standaard CRUD-operaties (*Create, Read, Update, Delete*).

In dit hoofdstuk gaan we deze code-duplicatie aanpakken. We introduceren `C# Generics` om een basisrepository te maken die bruikbaar is voor elke entiteit. Daarnaast introduceren we het `Unit of Work` patroon. Dit patroon zorgt ervoor dat we verschillende repositories kunnen beheren via één centrale toegangspoort en dat al onze database-acties binnen één enkele transactie gebeuren.

Terugblik & Focus

We hebben de overstap gemaakt naar een echte PostgreSQL-database. We hebben specifieke repositories gebouwd (zoals `LaptopRepository`) die asynchroon communiceren met de `DbContext` met behulp van LINQ.

We gaan in dit hoofdstuk onze datalaag naar een professioneel niveau tillen. We centraliseren gemeenschappelijke database-logica met behulp van Generics en beheren de data-transacties met het Unit of Work patroon.

C# Generics

Binnen de taal `C#` betekent 'generics' letterlijk "**niet beperkt tot één specifiek datatype**". Het stelt ons in staat om klassen, interfaces en methoden te schrijven waarbij we het specifieke datatype pas bepalen op het moment dat we de code daadwerkelijk gebruiken. Deze kunnen dan later gebruikt worden doorheen je programma-code waarbij het datatype flexibeler in gebruik is.

Info

💡 Boxing & Unboxing

Het grote voordeel, *naast niet constant dezelfde code hoeven te schrijven*, van generics is type-safety en performance. Zonder generics moesten we vaak werken met het basis-type object om flexibel te zijn. Dit vereist boxing (een waarde-type zoals int inpakken in een object) en unboxing (het object weer terugzetten). Dit kost veel geheugen en processorkracht. Generics vermijden dit proces volledig.

Probleem: Duplicate Code

Wanneer we kijken naar de repositories die we tot nu toe hebben geschreven, of *zouden* schrijven voor de nieuwe modellen, zien we een ongewenst patroon. Zie jij welk?

Methode	Code
LaptopRepository.GetAllAsync()	<code>_context.Laptops.ToListAsync()</code>
ProductRepository.GetAllAsync()	<code>_context.Producten.ToListAsync()</code>
KlantRepository.GetAllAsync()	<code>_context.Klanten.ToListAsync()</code>
LaptopRepository.GetByIdAsync(id)	<code>_context.Laptops.FindAsync(id)</code>
ProductRepository.GetByIdAsync(id)	<code>_context.Producten.FindAsync(id)</code>
KlantRepository.GetByIdAsync(id)	<code>_context.Klanten.FindAsync(id)</code>
LaptopRepository.AddAsync(entity)	<code>_context.Laptops.AddAsync(entity)</code>
ProductRepository.AddAsync(entity)	<code>_context.Producten.AddAsync(entity)</code>
KlantRepository.AddAsync(entity)	<code>_context.Klanten.AddAsync(entity)</code>

De code is vrijwel identiek, enkel de DbSet (*de tabel*) verschilt. Dit schendt het DRY-principe (*Don't Repeat Yourself*). Als we later beslissen om bijvoorbeeld te stoppen met Entity Framework en over te stappen naar Dapper, moeten we deze logica in *alle* specifieke repositories gaan aanpassen. Bovendien moeten we momenteel elke specifieke repository apart registreren in Program.cs en injecteren in onze controllers, wat leidt tot volle constructoren.

Tips

Onthou: Een goede programmeur is lui. We willen niet steeds opnieuw hetzelfde werk doen, we willen efficient zijn.

Oplossing: Generics<T>

We gaan een "base-repository" maken die alle standaard CRUD-acties in één keer implementeert voor een willekeurige entiteit `TEntity`.

Je werkt al de hele tijd met Generics!

Hoewel de term "Generics" misschien nieuw en complex in de oren klinkt, werk je er eigenlijk al lang onbewust mee. Denk maar eens aan de vertrouwde `List<T>`.

Wanneer je een lijst aanmaakt, schrijf je dat als volgt:

- `List<Laptop> mijnLaptops = new List<Laptop>();`
- `List<Product> mijnProducten = new List<Product>();`
- `List<string> namen = new List<string>();`

Die Laptop, Product en string zijn de **concrete datatypen** die je invult op de plaats van de `T` (de Type-parameter). `T` staat hier voor een placeholder. `T` wilt in principe zeggen: voeg hier je eigen data type mee en *tijdens runtime* zal `T` vervangen worden door het gekozen type.

Info

Stel je eens voor dat C# *geen* generics zou ondersteunen. Hoe zou een lijst er dan uitzien? De makers van C# hadden dan voor **elk nieuw model** een compleet nieuwe lijst-klasse moeten programmeren. Je had dan klassen moeten gebruiken zoals:

- LaptopList (die alleen laptops kan bevatten)
- ProductList (die alleen producten kan bevatten)
- StringList (die alleen tekst kan bevatten)

Als programmeur zou je telkens wanneer je een nieuw model aanmaakt (zoals Bestelling of Klant), ook een eigen BestellingList en KlantList klasse moeten schrijven om die objecten te kunnen groeperen. Dit zou onbegonnen werk zijn en leiden tot gigantische hoeveelheden gedupliceerde code. Dankzij Generics hebben de ontwikkelaars van Microsoft de logica

van een lijst (dingen toevoegen, verwijderen, sorteren) **één keer** universeel geschreven als `List<T>`. Het maakt de lijst niet uit wat erin gestopt wordt; jij bepaalt pas bij het initialiseren welk type erin gaat.

Exact hetzelfde principe passen we nu toe op onze `GenericRepository<TEntity>`. In plaats van een aparte repository te schrijven voor elk model, schrijven we de database-logica (ophalen, toevoegen, verwijderen) één keer universeel voor de placeholder `TEntity`.



Info

Zie de afbeelding hierboven. Wat verkies je? Een aparte pot aanschaffen, onderhouden en reinigen voor elk denkbaar type pasta, of één pot die voor elk type pasta gebruikt kan worden?

Generieke Repository aanmaken

Interface

We maken eerst een contract aan in de map `Repositories`. Voeg aan de `Repositories` folder een nieuwe interface `IGenericRepository.cs` toe.

```
1 namespace WebAPIDemo.Repositories csharp
2 {
3     // TEntity is een placeholder voor het uiteindelijke model (bv.
4     Laptop of Product)
5     public interface IGenericRepository<TEntity> where TEntity : class
6     {
7         // C - Create
8         void Add(TEntity entity);
9
10        // R - Read (All)
11        Task<IEnumerable<TEntity>> GetAllAsync();
12
13        // R - Read (Single)
14        Task<TEntity?> GetByIdAsync(int id);
15
16        // U - Update
17        void Update(TEntity entity);
18
19        // D - Delete
20        void Delete(TEntity entity);
21    }
22 }
```

Info

Merk op dat we de type-parameter <TEntity> gebruiken en een constraint (where TEntity : class) toevoegen om aan EF te vertellen dat TEntity altijd een klasse moet zijn en er dus niet zomaar een int of string met deze klasse gebruikt kan worden. Je kan deze evengoed <T> of iets anders noemen.

Implementerende klasse

Nu implementeren we deze interface. Omdat we niet weten over welke tabel het gaat, gebruiken we de methode `_context.Set<TEntity>()`. Hiermee vertellen we aan Entity Framework om de DbSet te gebruiken die hoort bij het type dat we op dat moment gebruiken. TEntity is in dit geval de placeholder die later vervangen zal worden.

Maak het bestand **GenericRepository.cs** aan in de map Repositories:

```
1 using Microsoft.EntityFrameworkCore;
2 using WebApiDemo.Data;
3
4 namespace WebAPIDemo.Repositories
5 {
6     public class GenericRepository : IGenericRepository
7     where TEntity : class
8     {
9         // _context wordt protected zodat overervende klassen deze ook
10        kunnen gebruiken.
11        protected readonly WebAPIDemoContext _context;
12
13        public GenericRepository(WebAPIDemoContext context)
14        {
15            _context = context;
16        }
17
18        public async Task<IEnumerable<TEntity>> GetAllAsync()
19        {
20            // Set<TEntity> pakt automatisch de juiste tabel (bv. Laptops
21            of Producten), gebaseerd op het type dat TEntity zal vervangen.
22            return await _context.Set<TEntity>().ToListAsync();
23        }
24
25        public async Task<TEntity?> GetByIdAsync(int id)
26        {
27            return await _context.Set<TEntity>().FindAsync(id);
28        }
29
30        public void Add(TEntity entity)
31        {
32            _context.Set<TEntity>().Add(entity);
33        }
34
35        public void Update(TEntity entity)
36        {
37            // Attach zorgt ervoor dat EF de entiteit gaat tracken als
38            'Modified'
39            _context.Set<TEntity>().Attach(entity);
40            _context.Entry(entity).State = EntityState.Modified;
41        }
42
43        public void Delete(TEntity entity)
```

csharp

```

43         {
44             _context.Set<TEntity>().Remove(entity);
        }
    }
}

```

Na het schrijven van deze klasse, hebben we een basisklasse die we kunnen gebruiken voor elk denkbaar type Repository. We kunnen nu met één klasse eender welke tabel aanspreken en er minstens een Get, GetAll, Add, Update en Delete op uitvoeren.

Notitie

Merk op dat de Add, Update en Delete methodes niet langer asynchroon (Task) zijn en geen SaveChanges() meer bevatten. **Dit is bewust.** EF Core Change Tracking houdt wijzigingen in het geheugen bij. Het daadwerkelijke wegschrijven naar de database gebeurt later centraal via de Unit of Work. Hierover later meer wanneer we data uit meerdere tabellen gelijktijdig willen manipuleren.

Specifieke Repositories

Soms is een generieke repository niet voldoende. Wat als we een specifieke methode nodig hebben, zoals `_laptopRepository.GetLaptopsByMerkAsync(string merk)`? Deze actie bestaat niet in de generieke basisklasse aangezien deze *té specifieke* logica bevat. In dat geval maken we nog steeds een specifieke repository, maar deze **erft over van de GenericRepository**. Zo kunnen we de basis-CRUD herbruiken en hoeven we **enkel de gespecialiseerde methodes te schrijven**.

ILaptopRepository.cs

We passen onze bestaande `ILaptopRepository.cs` interface aan zodat deze overerft van `IGenericRepository<Laptop>`. We hoeven de standaard CRUD-methoden hier nu niet meer te declareren.

```

1      using WebAPIDemo.Models;
2
3      namespace WebAPIDemo.Repositories
4      {
5          // We erven alle standaard methoden over voor het type Laptop
6          public interface ILaptopRepository : IGenericRepository<Laptop>
7

```

```
8      {
9          // We voegen enkel de specifieke methoden toe
10         Task<IEnumerable<Laptop>> GetLaptopsByMerkAsync(string merk);
11     }
12 }
```

LaptopRepository.cs

De implementatie erft over van `GenericRepository<Laptop>`. Via de constructor geven we de `DbContext` door aan de base repository. Ook hier hoeven we de herhalende methodes niet langer te schrijven.

```
1  using Microsoft.EntityFrameworkCore;
2  using WebApiDemo.Data;
3  using WebAPIDemo.Models;
4
5  namespace WebAPIDemo.Repositories
6  {
7      public class LaptopRepository : GenericRepository<Laptop>,
8      ILaptopRepository
9      {
10         // base(context) stuurt de binnengekomen context door naar de
11         GenericRepository
12         public LaptopRepository(WebAPIDemoContext context) :
13         base(context) { }
14
15         // We implementeren enkel de specifieke methode
16         public async Task<IEnumerable<Laptop>>
17         GetLaptopsByMerkAsync(string merk)
18         {
19             // Dankzij 'protected' in GenericRepository kunnen we hier
20             bij _context
21             return await _context.Laptops
22                 .Where(x => x.Merk.ToLower() == merk.ToLower())
23                 .ToListAsync();
24         }
25     }
26 }
```

Doe hetzelfde voor de BestellingRepository. Aangezien deze (nog) geen functionaliteit aanbiedt die niet door de GenericRepo aangeboden wordt, kunnen we deze klasse eenvoudig leeg laten en enkel van de correcte objecten laten inheriten.

IBestellingRepository.cs

```
1  using WebAPIDemo.Models;
2  using WebAPIDemo.Repositories;
3
4  namespace WebApiDemo.Repositories
5  {
6      public interface IBestellingRepository:
7      IGenericRepository<Bestelling>
8      {
9
10     }
11 }
```

csharp

BestellingRepository.cs

```
1  using WebApiDemo.Data;
2  using WebAPIDemo.Models;
3  using WebAPIDemo.Repositories;
4
5  namespace WebApiDemo.Repositories
6  {
7      public class BestellingRepository : GenericRepository<Bestelling>,
8      IBestellingRepository
9      {
10         public BestellingRepository(WebAPIDemoContext context):
11         base(context)
12         {
13
14         }
15     }
16 }
```

csharp

Unit Of Work

We hebben nu mooie (generieke) repositories, maar in onze controllers moeten we nog steeds voor elke entiteit een andere repository injecteren. Bijvoorbeeld in de ProductController injecteren we ILaptopRepository, maar in een toekomstige BestellingController zouden we IKlantRepository, IBestellingRepository, IProductRepository én IOrderLijnRepository moeten injecteren. Dit maakt de constructor **overvol en onoverzichtelijk**.

Daarnaast is er een groter probleem: Transactie-consistentie . Stel je voor dat je een bestelling plaatst. Je moet een record aanmaken in de Bestelling tabel én meerdere records in de OrderLijn tabel. Als de eerste SaveChanges() lukt, maar de computer crasht voordat de orderlijnen zijn opgeslagen, is je database inconsistent. Je hebt dan een bestelling zonder producten.

Het Unit of Work (UoW) patroon lost beide problemen op. Het fungeert als **één centrale toegangspoort tot al je repositories**. De Unit of Work beheert de DbContext en zorgt ervoor dat **alle wijzigingen** over verschillende repositories heen pas naar de database worden gestuurd *wanneer jij dat beslist via één centrale SaveChangesAsync()* aanroep.

Interface

Maak in de **Data** folder een nieuwe interface aan IUnitOfWork.cs . Hieraan voegen we elke Repository toe dat onze applicatie gebruikt en een methode om alle wijzigingen in 1 keer op te slaan.

```
1  using WebAPIDemo.Models;                                     csharp
2
3  namespace WebAPIDemo.Repositories
4  {
5      public interface IUnitOfWork
6      {
7          // Specifieke repository omdat deze extra methoden heeft
8          ILaptopRepository LaptopRepository { get; }
9
10         // Generieke repositories voor entiteiten die enkel basis-CRUD
11         nodig hebben
12         IGenericRepository<Product> ProductRepository { get; }
13         IGenericRepository<Klant> KlantRepository { get; }
```

```
14         IGenericRepository<Bestelling> BestellingRepository { get; }
15         IGenericRepository<OrderLijn> OrderLijnRepository { get; }
16
17         // De centrale Save methode (asynchroon)
18         Task<int> SaveChangesAsync();
19     }
}
```

UnitOfWork.cs

De UnitOfWork klasse krijgt de DbContext via constructor injection binnen. Het is de verantwoordelijkheid van de UnitOfWork om deze context door te geven aan alle repositories die hij beheert. Zo werken alle repositories met *exact hetzelfde geheugenbeeld* van de database.

We gebruiken een techniek genaamd *Lazy Loading* (luie initialisatie) voor de repositories. *We maken ze pas aan op het moment dat ze voor het eerst worden opgevraagd*. Dat bespaart geheugen en onnodige cpu cycles.

Maak het bestand `UnitOfWork.cs` aan in de map **Data**:

```
1  using WebApiDemo.Data;
2  using WebAPIDemo.Models;
3
4  namespace WebAPIDemo.Repositories
5  {
6      public class UnitOfWork : IUnitOfWork
7      {
8          private readonly WebAPIDemoContext _context;
9
10         // Private fields voor de repositories (voor lazy loading)
11         private ILaptopRepository? _laptopRepository;
12         private IGenericRepository<Product>? _productRepository;
13         private IGenericRepository<Klant>? _klantRepository;
14         private IGenericRepository<Bestelling>? _bestellingRepository;
15         private IGenericRepository<OrderLijn>? _orderLijnRepository;
16
17         public UnitOfWork(WebAPIDemoContext context)
18         {
19             _context = context;
20         }
21     }
```

```

22         // Lazy Loading properties: Als een repo null is, maak hem dan
23         aan.
24         // We gebruiken de null-coalescing assignment operator (??=).
25         // Aangezien LaptopRepository specifieke methodes bevat, kiezen
26         we voor een specifiek type en geen GenericRepository
27         public ILaptopRepository LaptopRepository =>
28             _laptopRepository ??= new LaptopRepository(_context);
29
30         // Voor de generieke repositories maken we direct een
31         GenericRepository<T> aan
32         public IGenericRepository<Product> ProductRepository =>
33             _productRepository ??= new GenericRepository<Product>
34             (_context);
35
36         public IGenericRepository<Klant> KlantRepository =>
37             _klantRepository ??= new GenericRepository<Klant>(_context);
38
39         public IGenericRepository<Bestelling> BestellingRepository =>
40             _bestellingRepository ??= new GenericRepository<Bestelling>
41             (_context);
42
43         public IGenericRepository<OrderLijn> OrderLijnRepository =>
44             _orderLijnRepository ??= new GenericRepository<OrderLijn>
45             (_context);
46
47         // De centrale plek waar SaveChangesAsync wordt aangeroepen
48         public async Task<int> SaveChangesAsync()
49         {
50             return await _context.SaveChangesAsync();
51         }
52     }

```

Tips

Een Unit Of Work past mooi binnen het Single Responsibility principle (SRP) .("A class should have only one reason to change)" De enige reden waarom deze klasse/ Interface ooit aangepast zal moeten worden is omdat er iets verandert aan de Database structuur zoals een extra Repository die toegevoegd of verwijderd werd, of een GenericRepo die vervangen wordt door een gespecialiseerde variant.

[Single Responsibility op Wikipedia](https://en.wikipedia.org/wiki/Single_responsibility_principle)

Unit Of Work registreren en gebruiken

Doordat de UnitOfWork de repositories beheert, hoeven we in Program.cs niet langer elke repository apart te registreren. We registreren enkel nog de DbContext en de IUnitOfWork.

Program.cs

```
1  using Microsoft.EntityFrameworkCore;
2  using Scalar.AspNetCore;
3  using WebApiDemo.Data;
4  using WebAPIDemo.Repositories;
5
6  var builder = WebApplication.CreateBuilder(args);
7
8  // Add services to the container.
9
10 builder.Services.AddControllers();
11 // Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
12 builder.Services.AddOpenApi();
13
14 // Registreer UnitOfWork
15 builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
16
17 // Registreer de Database Connectie
18 var connectionString =
19 builder.Configuration.GetConnectionString("PostgresConnection");
20
21 // Registreer de DbContext met de PostgreSQL provider (Npgsql)
22 builder.Services.AddDbContext<WebAPIDemoContext>(options =>
23     options.UseNpgsql(connectionString));
24
25 var app = builder.Build();
26
27 // Configure the HTTP request pipeline.
28 if (app.Environment.IsDevelopment())
29 {
30     app.MapOpenApi();
31     app.MapScalarApiReference(); // <-- Voeg dit toe voor de interactieve
32     UI
33 }
```

csharp

```
34
35 app.UseHttpsRedirection();
36 app.UseAuthorization();
   app.MapControllers();
   app.Run();
```

ProductController.cs

In onze controller injecteren we nu de **IUnitOfWork** in plaats van de specifieke **repository(ies)**. Via de **UnitOfWork** hebben we toegang tot alle data.

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebApiDemo.Repositories;
3  using WebAPIDemo.Models;
4  using WebAPIDemo.Repositories;
5  // Zorg ervoor dat je Microsoft.Extensions.Logging hebt via using
6
7  namespace WebAPIDemo.Controllers;
8
9  [ApiController]
10 [Route("api/[controller]")]
11 public class ProductController : ControllerBase
12 {
13     // Onze "ingrediënten" (Dependencies)
14     private readonly IUnitOfWork _uow;
15     private readonly ILogger<ProductController> _logger;
16
17     // Vraag om en injecteer Dependencies
18     public ProductController(IUnitOfWork uow, ILogger<ProductController>
19 logger)
20     {
21         _uow = uow;
22         _logger = logger;
23     }
24
25     [HttpGet]
26     public async Task<ActionResult<List<Laptop>>> GetAlleLaptopsAsync()
27     {
28         _logger.LogInformation("GET request ontvangen voor alle
29 laptops.");
30         return Ok(await _uow.LaptopRepository.GetAllAsync());
31     }
```

```
32     [HttpGet("{id}")]
33     public async Task<ActionResult<Laptop>> GetLaptopByIdAsync(int id)
34     {
35         _logger.LogInformation($"GET request voor laptop met ID: {id}");
36
37         Laptop? laptop = await _uow.LaptopRepository.GetByIdAsync(id);
38
39         if (laptop == null)
40         {
41             _logger.LogWarning($"Laptop met ID {id} werd niet
42 gevonden.");
43             return NotFound($"Helaas, we konden geen laptop vinden met Id
44 {id}.");
45         }
46
47         return Ok(laptop);
48     }
49
50     [HttpPost]
51     public async Task<ActionResult<Laptop>> CreateLaptopAsync(Laptop
52 nieuweLaptop)
53     {
54         _logger.LogInformation($"Aanmaken van nieuwe laptop:
55 {nieuweLaptop.Merk}");
56
57         // 1. Voeg de actie toe aan de Change Tracker via de repository
58         _uow.LaptopRepository.Add(nieuweLaptop);
59
60         // 2. Schrijf de wijzigingen daadwerkelijk weg via de Unit of
61 Work
62         await _uow.SaveChangesAsync();
63
64         return CreatedAtAction(nameof(CreateLaptopAsync), new { id =
65 nieuweLaptop.Id }, nieuweLaptop);
66     }
67
68     [HttpPut("{id}")]
69     public async Task<ActionResult> UpdateLaptopAsync(int id, Laptop
70 bijgewerkteLaptop)
71     {
72         _logger.LogInformation($"Update request voor laptop met ID:
73 {id}");
74
```

```
75         if (id != bijgewerkteLaptop.Id)
76         {
77             return BadRequest();
78         }
79
80         Laptop? bestaandeLaptop = await
81         _uow.LaptopRepository.GetByIdAsync(id);
82
83         if (bestaandeLaptop == null)
84         {
85             _logger.LogWarning($"Update mislukt: Laptop met ID {id}
86 bestaat niet.");
87             return NotFound($"Kan geen laptop bijwerken met Id {id},
88 omdat deze niet bestaat.");
89         }
90
91         _uow.LaptopRepository.Update(bijgewerkteLaptop);
92         await _uow.SaveChangesAsync();
93
94         _logger.LogInformation($"Laptop met ID {id} is succesvol
95 bijgewerkt.");
96         return NoContent();
97     }
98
99     [HttpDelete("{id}")]
100     public async Task<ActionResult> DeleteLaptopAsync(int id)
101     {
102         _logger.LogInformation($"Delete request voor laptop met ID:
103 {id}");
104
105         Laptop? laptop = await _uow.LaptopRepository.GetByIdAsync(id);
106
107         if (laptop == null)
108         {
109             _logger.LogError($"Delete mislukt: Laptop met ID {id} werd
110 niet gevonden.");
111             return NotFound($"Kan laptop met Id {id} niet verwijderen,
112 omdat deze niet is gevonden.");
113         }
114
115         _uow.LaptopRepository.Delete(laptop);
116         await _uow.SaveChangesAsync();
117     }
118 }
```

```
        _logger.LogInformation($"Laptop met ID {id} is succesvol  
verwijderd.");  
        return NoContent();  
    }  
}
```

Info

De focus van de Controller

Merk op dat de verantwoordelijkheid van de controller nu nog duidelijker is: hij handelt het HTTP-request af, valideert de input (ModelState), stuurt de opdracht door naar de Unit of Work, en geeft de juiste HTTP-statuscode terug. Hij bemoeit zich totaal niet meer met de interne werking van de database.

Het probleem van de losse queries (N+1 probleem)

Onze `GenericRepository<TEntity>` is fantastisch voor het uitvoeren van standaard CRUD-operaties op één specifieke tabel. Maar wat gebeurt er als we data uit **meerdere gerelateerde tabellen** nodig hebben?

Stel dat we een controller-endpoint willen maken dat een `Bestelling` ophaalt, inclusief de bijbehorende `Klant` en de bestelde `Producten`.

Notitie

Onderstaande is louter ter illustratie en hoeft niet overgeschreven te worden. Sterker nog, het is zelfs een heel slecht idee om verschillende queries gelijktijdig te versturen!

Een oplossing?: Meerdere losse queries uitvoeren?

Aangezien onze generieke repository geen kennis heeft van relaties, zouden we in de controller genoodzaakt zijn om meerdere losse requests naar de database te sturen:

```
1 [HttpGet("{id}/handmatig")]                                     csharp  
2 public async Task<ActionResult> GetBestellingHandmatigAsync(int id)  
3 {  
4     // Query 1: Haal de bestelling op  
5     Bestelling? bestelling = await
```

```
6  _uow.BestellingRepository.GetByIdAsync(id);
7      if (bestelling == null) return NotFound();
8
9      // Query 2: Haal handmatig de bijbehorende klant op
10     Klant? klant = await
11     _uow.KlantRepository.GetByIdAsync(bestelling.KlantId);
12     bestelling.Klant = klant;
13
14     // Query 3: Haal alle orderlijnen op voor deze bestelling
15     IEnumerable<OrderLijn> alleLijnen = await
16     _uow.OrderLijnRepository.GetAllAsync();
17     bestelling.Orderlijnen = alleLijnen.Where(o => o.BestellingId ==
18     id).ToList();
19
20     // Query 4, 5, 6... Voor elke orderlijn ook nog het product ophalen?
21     foreach (var lijn in bestelling.Orderlijnen)
22     {
23         lijn.Product = await
24         _uow.ProductRepository.GetByIdAsync(lijn.ProductId);
25     }
26
27     return Ok(bestelling);
28 }
```

Waarom is dit een slecht idee?

Hoewel de bovenstaande code functioneel werkt, veroorzaakt het een ernstig performance-probleem:

- **N+1 Query Problem** : Voor één enkele bestelling voeren we soms wel 10 tot 20 losse netwerkverzoeken uit naar onze PostgreSQL-database. Als je een lijst van 50 bestellingen wilt tonen, vuurt je applicatie honderden queries af.
- **Trage API** : Elke SQL-query kost netwerktime (latency). Je API wordt merkbaar traag.
- **Controller vervuiling** : De controller-methode staat vol met database-combinatiel logica, wat het Single Responsibility Principle (SRP) zwaar schendt.
- **NULL Properties** : Als we de gerelateerde data niet handmatig ophalen, blijven de properties `bestelling.Klant` en `bestelling.Orderlijnen` simpelweg null, omdat Entity Framework deze standaarden uit veiligheid en performance niet meestuurt.

Hoe lossen we dit op? We hebben een manier nodig om aan de database te zeggen: "Geef mij Bestelling X, en JOIN in één enkele query meteen de Klant, Orderlijnen en Producten mee."

Hiervoor moeten we onze GenericRepository uitbreiden met gespecialiseerde methodes en gebruikmaken van Eager Loading (`.Include()`). En dat is precies wat we gaan behandelen in het volgende hoofdstuk!

Samenvatting

- **Generics<T>** : Stellen ons in staat om herbruikbare code-componenten (zoals interfaces en klassen) te schrijven die niet gebonden zijn aan één datatype, wat duplicatie voorkomt en type-safety garandeert.
- **Generieke Repository** : Een basisklasse die standaard CRUD-operaties één keer implementeert voor een willekeurige entiteit TEntity, met behulp van `_context.Set<TEntity>()`.
- **Specifieke Repository** : Een klasse die overerft van GenericRepository voor de basis CRUD, maar wordt uitgebreid met specifieke methoden (bijv. zoeken op merk) voor een bepaald model.
- **Loose Coupling** : Doordat controllers enkel communiceren met interfaces, blijft de code flexibel. We kunnen de onderliggende database-implementatie later aanpassen zonder de controllers aan te raken.
- **Unit of Work Patroon(UOW)** : Een centrale toegangspoort die alle repositories beheert. Het zorgt voor een opgeruimde constructor in de controller en garandeert dat alle database-acties binnen één enkele transactie gebeuren via een centrale `SaveChangesAsync()`.
- **N+1 Query Probleem** : Een performance-probleem waarbij gerelateerde data via meerdere losse databasequeries wordt opgehaald in plaats van via één gecombineerde query (zoals met `.Include()`), wat leidt tot hoge netwerk-latency en trage API's.

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeveTM